FPT UNIVERSITY

LÊ HUỲNH KHÁNH HUY
LÊ MINH TOÀN
LĂNG ANH ĐỨC
LÊ NGUYỄN NHẬT HUY
NGUYỄN TẤN TÀI
PHẠM NHƯ LONG PHÚ

BÁO CÁO ĐỒ ÁN MÔN HỌC

THIẾT KẾ VÀ MÔ PHỎNG BỘ NHỚ ĐỆM CACHE ÁNH XẠ TRỰC TIẾP BẰNG RTL

CACHE MEMORY SIMULATOR RTL: DESIGN AND SIMULATION OF A DIRECT-MAPPED CACHE

MÔN HỌC: FAP201 - IC2007

GIẢNG VIÊN HƯỚNG DẪN
Mr.Vinh

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm em xin gửi lời cảm ơn chân thành đến quý thầy cô Trường Đại học Công nghệ Thông tin – Đại học Quốc gia TP. Hồ Chí Minh, đặc biệt là quý thầy cô Khoa Kỹ thuật Máy tính, đã tận tình truyền đạt những kiến thức nền tảng về kiến trúc máy tính và thiết kế hệ thống số, giúp nhóm có đủ cơ sở để thực hiện đề tài này.

Nhóm em xin gửi lời cảm ơn sâu sắc đến thầy Vinh đã hướng dẫn, góp ý và định hướng cho nhóm trong suốt quá trình thực hiện đề tài "Cache Memory Simulator RTL".

Trong quá trình thực hiện, dù đã cố gắng hoàn thiện, báo cáo khó tránh khỏi những thiếu sót. Nhóm em rất mong nhận được những ý kiến đóng góp của quý thầy cô để đề tài được hoàn thiện hơn.

Nhóm em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 11 năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 1

1.1. Đặt vấn đề 1

1.2. Mục tiêu thực hiện đề tài 2

1.3. Đối tượng và phạm vi nghiên cứu 2

1.4. Phương pháp thực hiện 4

1.5. Kết quả mong đợi 4

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 6

2.1. Tổng quan về bộ nhớ máy tính 6

2.2. Khái niệm Cache Memory 7

2.3. Nguyên lý hoạt động của Cache 7

2.4. Cache Hit và Cache Miss 8

2.5. Các phương pháp ánh xạ Cache 8

2.6. Các chính sách ghi dữ liệu 9

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 11

3.1. Phân tích yêu cầu hệ thống 11

3.2. Kiến trúc tổng thể hệ thống 11

3.3. Phân tích chức năng các module RTL 12

3.4. Mô tả tín hiệu giao tiếp hệ thống 12

3.5. Luồng dữ liệu và thực thi hệ thống 13

CHƯƠNG 4. TRIỂN KHAI THIẾT KẾ RTL 16

4.1. Môi trường và công cụ thiết kế 16

4.2. Thiết kế module Cache Controller 16

4.3. Thiết kế module Cache Data Array 16

4.4. Thiết kế các module hỗ trợ 18

4.5. Kết nối các module tại Cache Top 18

4.6. Nguyên lý hoạt động của hệ thống 19

4.7. Kết luận chương 19

CHƯƠNG 5. MÔ PHỎNG VÀ KIỂM CHỨNG 20

5.1. Phương pháp kiểm chứng 20

5.2. Các trường hợp kiểm thử 20

5.3. Kết quả mô phỏng 21

5.4. Phân tích dạng sóng tín hiệu (Waveform) 22

5.5. Đánh giá kết quả 22

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN24

6.1. Những kết quả đạt được24

6.2. Những hạn chế26

6.3. Hướng phát triển trong tương lai26

6.4. Kết luận chung27

TÀI LIỆU THAM KHẢO29

DANH MỤC HÌNH ẢNH

Hình 2.1. Hệ thống phân cấp bộ nhớ trong máy tính 6
Hình 2.2. Tổ chức và cơ chế kiểm tra của Cache Direct-mapped trong đề tài 7
Hình 3.1. Sơ đồ khối tổng thể của hệ thống Cache Memory Simulator 12
Hình 3.2. Lưu đồ xử lý yêu cầu đọc/ghi của hệ thống 14
Hình 5.1. Kết quả mô phỏng module Cache Data Array trong cửa sổ Transcript 21
Hình 5.2. Kết quả mô phỏng khối Cache Storage trên ModelSim 21
Hình 5.3. Dạng sóng tín hiệu của module Cache Data Array 22

DANH MỤC BẢNG

Bảng 1.1. Thông số cấu hình chính của hệ thống Cache3
Bảng 2.1. Các thuật ngữ cơ bản của Cache7
Bảng 2.2. Phân loại Cache Miss theo mô hình 3C8
Bảng 2.3. So sánh các phương pháp ánh xạ Cache 9
Bảng 2.4. Các tổ hợp chính sách ghi phổ biến 10
Bảng 3.1. Thông số thiết kế của hệ thống 11
Bảng 3.2. Tín hiệu giao tiếp phía CPU12
Bảng 3.3. Tín hiệu giao tiếp phía bộ nhớ chính 13
Bảng 3.4. Ví dụ thực thi chuỗi truy xuất mẫu 14
Bảng 4.1. Các trạng thái của FSM trong Cache Controller 16
Bảng 4.2. Thông số thiết kế phiên bản hiện tại của Cache Data Array 17
Bảng 4.3. Tín hiệu giao tiếp của module cache data_array 17
Bảng 5.1. Các trường hợp kiểm thử của hệ thống 20
Bảng 5.2. Các trường hợp kiểm thử module Cache Data Array 21
Bảng 6.1. Kết quả phân tích chuỗi truy xuất mẫu trên Cache Direct-mapped 25
Bảng 6.2. Đối chiếu kết quả đạt được với mục tiêu đề tài 25
Bảng 6.3. So sánh dự kiến giữa Direct-mapped và 2-way Set Associative 27

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
AMAT	Average Memory Access Time – Thời gian truy xuất bộ nhớ trung bình
CPU	Central Processing Unit – Bộ xử lý trung tâm
DRAM	Dynamic Random Access Memory
FIFO	First In First Out
FPGA	Field Programmable Gate Array
FSM	Finite State Machine – Máy trạng thái hữu hạn
HDL	Hardware Description Language – Ngôn ngữ mô tả phần cứng
LRU	Least Recently Used
RTL	Register Transfer Level – Mức truyền thanh ghi
SRAM	Static Random Access Memory
UVM	Universal Verification Methodology

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Đặt vấn đề

Trong kiến trúc máy tính hiện đại, hiệu năng của hệ thống không chỉ phụ thuộc vào tốc độ xử lý của bộ vi xử lý (CPU) mà còn phụ thuộc đáng kể vào tốc độ truy xuất dữ liệu từ bộ nhớ. Trong quá trình hoạt động, CPU thường xuyên phải đọc dữ liệu và lệnh từ bộ nhớ chính (Main Memory). Tuy nhiên, tốc độ truy xuất của bộ nhớ chính thường thấp hơn tốc độ xử lý của CPU, dẫn đến tình trạng CPU phải chờ dữ liệu trong quá trình thực thi chương trình.

Để khắc phục vấn đề này, bộ nhớ đệm (Cache Memory) được sử dụng như một tầng bộ nhớ trung gian giữa CPU và bộ nhớ chính. Cache có nhiệm vụ lưu trữ tạm thời các dữ liệu và lệnh được truy cập thường xuyên hoặc gần đây, giúp CPU có thể truy xuất dữ liệu nhanh hơn so với việc truy cập trực tiếp vào bộ nhớ chính.

Hiệu quả hoạt động của Cache được đánh giá thông qua các khái niệm như Cache Hit, Cache Miss và Hit Rate. Khi dữ liệu cần truy xuất đã tồn tại trong Cache, hệ thống xảy ra Cache Hit. Ngược lại, khi dữ liệu chưa được lưu trữ trong Cache, hệ thống xảy ra Cache Miss và cần thực hiện quá trình lấy dữ liệu từ bộ nhớ chính. Bên cạnh đó, khi Cache không còn vị trí lưu trữ trống cho một block mới, hệ thống cần thực hiện cơ chế thay thế dữ liệu (Replacement).

Trong lĩnh vực thiết kế phần cứng số, việc xây dựng và mô phỏng một hệ thống Cache ở mức RTL (Register Transfer Level) là phương pháp phù hợp để nghiên cứu cấu trúc, nguyên lý hoạt động và sự phối hợp giữa các khối chức năng trong hệ thống bộ nhớ. Thông qua quá trình thiết kế và kiểm chứng, người thực hiện có thể hiểu rõ hơn cách một yêu cầu truy xuất bộ nhớ được tiếp nhận, xử lý và phản hồi.

Xuất phát từ những kiến thức đã tìm hiểu về kiến trúc máy tính và thiết kế phần cứng số, nhóm lựa chọn đề tài “Cache Memory Simulator RTL” nhằm xây dựng một mô hình bộ nhớ đệm có quy mô nhỏ, sử dụng phương pháp ánh xạ trực tiếp (Direct-mapped Cache). Hệ thống được tổ chức với 4 sets, mỗi block có khả năng lưu trữ 2 words. Thông qua các hoạt động truy xuất bộ nhớ được tạo bởi Testbench, nhóm tiến hành quan sát và đánh giá quá trình hoạt động của Cache, bao gồm Cache Hit, Cache Miss, Hit Rate và cơ chế thay thế dữ liệu.

Đề tài không chỉ giúp củng cố kiến thức lý thuyết về tổ chức bộ nhớ mà còn tạo cơ hội vận dụng kiến thức thiết kế RTL vào việc xây dựng, mô phỏng và kiểm chứng một hệ thống phần cứng số có cấu trúc rõ ràng.

1.2. Mục tiêu thực hiện đề tài

1.2.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và mô phỏng một hệ thống Cache Memory có quy mô nhỏ bằng RTL, sử dụng kiến trúc Direct-mapped. Qua đó, tìm hiểu nguyên lý hoạt động của bộ nhớ đệm, cách tổ chức dữ liệu, phương thức ánh xạ địa chỉ và cơ chế xử lý các yêu cầu truy xuất bộ nhớ.

Đồng thời, đề tài hướng đến việc kiểm chứng hoạt động của hệ thống thông qua Testbench, đánh giá khả năng xử lý các trường hợp Cache Hit và Cache Miss, tính toán tỷ lệ Cache Hit và quan sát quá trình thay thế dữ liệu khi xảy ra xung đột ánh xạ.

1.2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, nhóm tập trung thực hiện các nội dung sau:

- **Nghiên cứu cơ sở lý thuyết:** tìm hiểu kiến trúc bộ nhớ máy tính, vai trò của Cache Memory, nguyên lý hoạt động, các phương pháp ánh xạ địa chỉ và những khái niệm liên quan đến Cache Hit, Cache Miss.
- **Xây dựng kiến trúc Cache:** thiết kế mô hình Cache Direct-mapped với 4 sets và kích thước mỗi block là 2 words.
- **Thiết kế các khối chức năng:** xây dựng và kết nối các module RTL để mô phỏng hoạt động của hệ thống Cache, đảm bảo các khối có thể phối hợp trong quá trình xử lý yêu cầu truy xuất bộ nhớ.
- **Xây dựng môi trường kiểm thử:** sử dụng Testbench để tạo các yêu cầu truy xuất bộ nhớ với những địa chỉ khác nhau, phục vụ việc kiểm tra hoạt động của hệ thống.
- **Phân tích kết quả truy xuất:** theo dõi và xác định các trường hợp Cache Hit, Cache Miss, đồng thời tính toán tỷ lệ Hit Rate dựa trên số lần truy xuất.
- **Kiểm tra cơ chế thay thế:** quan sát hoạt động của Cache khi những block dữ liệu khác nhau được ánh xạ vào cùng một set, từ đó đánh giá khả năng xử lý xung đột dữ liệu của kiến trúc Direct-mapped.

- **Đánh giá hệ thống:** phân tích kết quả mô phỏng, nhận xét những ưu điểm, hạn chế và đề xuất các hướng phát triển phù hợp.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống bộ nhớ đệm Cache Memory được thiết kế và mô phỏng bằng RTL. Các thành phần và nội dung chính được tập trung nghiên cứu bao gồm:

- Kiến trúc bộ nhớ Cache và mối liên hệ giữa CPU, Cache và bộ nhớ chính.
- Phương pháp ánh xạ trực tiếp (Direct-mapped).
- Cấu trúc tổ chức dữ liệu gồm các Set và Block.
- Cơ chế xác định Cache Hit và Cache Miss, cách tính toán tỷ lệ Hit Rate.
- Cơ chế thay thế block dữ liệu khi xảy ra xung đột ánh xạ.
- Hoạt động phối hợp giữa các module RTL và phương pháp kiểm chứng thông qua Testbench.

1.3.2. Phạm vi nghiên cứu

Trong phạm vi đề tài, nhóm tập trung xây dựng và mô phỏng một hệ thống Cache có quy mô nhỏ với các thông số cấu hình chính được trình bày trong Bảng 1.1.

Bảng 1.1. Thông số cấu hình chính của hệ thống Cache

Thông số	Cấu hình
Kiến trúc Cache	Direct-mapped
Số lượng Set	4
Số lượng Block trong mỗi Set	1
Kích thước mỗi Block	2 words
Tổng số Cache Line	4
Phương thức kiểm thử	Memory Access Testbench

Về chức năng, hệ thống tập trung vào các nội dung sau:

- **Xử lý yêu cầu truy xuất bộ nhớ:** Testbench tạo ra các yêu cầu đọc dữ liệu tại những địa chỉ bộ nhớ khác nhau. Hệ thống tiếp nhận địa chỉ, xác định vị trí Set tương ứng và kiểm tra dữ liệu có tồn tại trong Cache hay không.

- **Xác định Cache Hit và Cache Miss:** dựa trên thông tin địa chỉ và trạng thái lưu trữ của Cache, hệ thống xác định yêu cầu truy xuất thuộc trường hợp Hit hay Miss. Các kết quả này được sử dụng để đánh giá hiệu quả hoạt động của bộ nhớ đệm.
- **Thống kê Hit Rate:** thông qua số lượng yêu cầu truy xuất thành công và tổng số yêu cầu truy xuất, hệ thống đánh giá tỷ lệ Cache Hit theo công thức (1.1).

  Hit Rate = (Số lần Cache Hit / Tổng số lần truy xuất) × 100% *(1.1)*
- **Xử lý thay thế dữ liệu:** đối với kiến trúc Direct-mapped, mỗi block dữ liệu trong bộ nhớ chính chỉ được ánh xạ đến một vị trí xác định trong Cache. Khi một block mới được truy xuất và ánh xạ đến một Set đang chứa một block khác, dữ liệu cũ tại vị trí đó sẽ được thay thế.

Về phạm vi kiểm chứng, nhóm sử dụng các chuỗi truy xuất địa chỉ bộ nhớ để quan sát hoạt động của hệ thống. Một chuỗi ví dụ được xác định trong đề tài gồm các yêu cầu đọc tại địa chỉ 0x10, 0x11, 0x20 và 0x10.

Đề tài tập trung vào mô hình Cache Direct-mapped. Việc nghiên cứu và so sánh với kiến trúc 2-way Set Associative được xem là hướng mở rộng nhằm đánh giá sự khác biệt giữa các phương pháp tổ chức Cache, không được xem là chức năng đã hoàn thành nếu chưa có kết quả triển khai và kiểm chứng tương ứng.

1.4. Phương pháp thực hiện

Để xây dựng và hoàn thiện đề tài, nhóm tiến hành theo các phương pháp sau:

- **Nghiên cứu lý thuyết:** tìm hiểu các tài liệu liên quan đến kiến trúc máy tính, nguyên lý hoạt động của bộ nhớ Cache, phương pháp ánh xạ trực tiếp, cách phân chia địa chỉ bộ nhớ và các cơ chế xử lý khi xảy ra Cache Hit hoặc Cache Miss. Từ đó, nhóm xác định các yêu cầu chức năng và thông số cấu hình cần thiết cho hệ thống.
- **Phân tích và thiết kế hệ thống:** dựa trên yêu cầu đề tài, nhóm phân tích kiến trúc tổng thể, xác định các khối chức năng, tín hiệu giao tiếp và mối quan hệ giữa các module, sau đó xây dựng sơ đồ khối thể hiện cấu trúc và luồng hoạt động của hệ thống.

- **Thiết kế RTL:** triển khai các khối chức năng bằng ngôn ngữ mô tả phần cứng Verilog HDL theo cấu trúc đã phân tích. Các module được thiết kế với nhiệm vụ riêng biệt và được kết nối để tạo thành hệ thống hoàn chỉnh.
- **Mô phỏng và kiểm chứng:** xây dựng Testbench để tạo các yêu cầu truy xuất bộ nhớ với những địa chỉ khác nhau. Thông qua quá trình mô phỏng, nhóm theo dõi các tín hiệu và trạng thái hoạt động của Cache để xác định tính đúng đắn của thiết kế.
- **Phân tích và đánh giá:** thu thập kết quả mô phỏng, thống kê số lần Cache Hit và Cache Miss, tính toán Hit Rate và phân tích những trường hợp xảy ra thay thế dữ liệu, từ đó đưa ra nhận xét về kết quả đạt được cũng như những hạn chế còn tồn tại.

1.5. Kết quả mong đợi

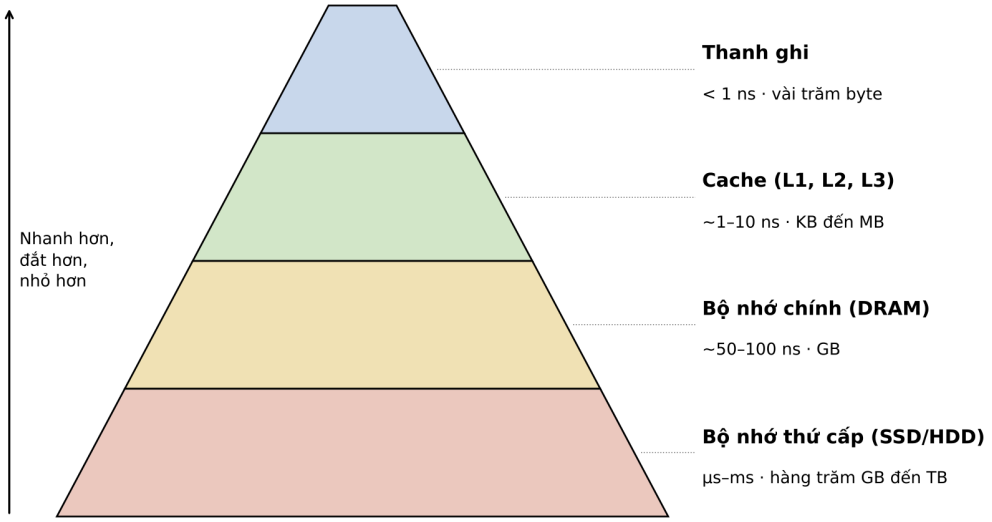
Sau khi hoàn thành, đề tài hướng đến các kết quả sau:

- Bản đặc tả kiến trúc hoàn chỉnh cho hệ thống Cache Direct-mapped 4 line, mỗi line 2 byte, bao gồm cách phân chia địa chỉ, chính sách ghi và các tín hiệu giao tiếp.
- Mã nguồn Verilog HDL của các module: Cache Controller, Cache Data Array, khối lưu Tag/Valid, Main Memory, bộ giải mã địa chỉ, bộ đếm hiệu năng và module Cache Top.
- Các Testbench tự kiểm tra (in kết quả PASS/FAIL) cho từng module và cho toàn hệ thống.
- Kết quả mô phỏng trên ModelSim (Transcript, Waveform) chứng minh hệ thống xác định đúng Cache Hit, Cache Miss, thực hiện thay thế dữ liệu và tính được Hit Rate cho các chuỗi truy xuất mẫu.
- Báo cáo tổng hợp, đánh giá ưu nhược điểm của kiến trúc Direct-mapped và đề xuất hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về bộ nhớ máy tính

Bộ nhớ là thành phần dùng để lưu trữ chương trình và dữ liệu mà CPU xử lý. Một hệ thống lý tưởng cần bộ nhớ vừa nhanh, vừa lớn, vừa rẻ; tuy nhiên trong thực tế, các công nghệ bộ nhớ nhanh (như SRAM) có giá thành cao và mật độ thấp, còn các công nghệ có dung lượng lớn (như DRAM, SSD, HDD) lại chậm hơn nhiều. Vì vậy, máy tính hiện đại tổ chức bộ nhớ theo dạng phân cấp (Memory Hierarchy), như minh họa ở Hình 2.1.



Hình 2.1. Hệ thống phân cấp bộ nhớ trong máy tính

Ở đỉnh phân cấp là các thanh ghi bên trong CPU với tốc độ cao nhất nhưng số lượng rất ít. Tiếp theo là các mức Cache (L1, L2, L3) được chế tạo bằng SRAM, đặt gần hoặc nằm trong chip xử lý. Bên dưới là bộ nhớ chính sử dụng DRAM và cuối cùng là bộ nhớ thứ cấp như SSD, HDD. Càng xuống thấp, dung lượng càng lớn, giá thành mỗi bit càng rẻ nhưng thời gian truy xuất càng tăng.

Hệ thống phân cấp bộ nhớ hoạt động hiệu quả nhờ nguyên lý cục bộ (Principle of Locality) của chương trình:

- **Cục bộ thời gian (Temporal Locality):** một vị trí bộ nhớ vừa được truy cập có khả năng cao sẽ được truy cập lại trong thời gian ngắn, ví dụ các biến đếm và lệnh trong vòng lặp.

- **Cục bộ không gian (Spatial Locality):** khi một vị trí bộ nhớ được truy cập, các vị trí lân cận cũng có khả năng cao sẽ được truy cập tiếp theo, ví dụ khi duyệt mảng hoặc thực thi lệnh tuần tự.

Nhờ hai tính chất này, chỉ cần giữ một phần nhỏ dữ liệu đang được sử dụng ở mức bộ nhớ nhanh, CPU đã có thể phục vụ phần lớn các yêu cầu truy xuất mà không phải chờ bộ nhớ chậm.

2.2. Khái niệm Cache Memory

Cache Memory là bộ nhớ có dung lượng nhỏ, tốc độ cao, nằm giữa CPU và bộ nhớ chính, dùng để lưu bản sao của các block dữ liệu thường được truy cập. Dữ liệu được trao đổi giữa Cache và bộ nhớ chính theo đơn vị block (khối) gồm một số word liên tiếp, nhờ đó tận dụng được tính cục bộ không gian. Mỗi vị trí chứa một block trong Cache được gọi là một Cache Line. Bảng 2.1 tóm tắt các thuật ngữ cơ bản.

Bảng 2.1. Các thuật ngữ cơ bản của Cache

Thuật ngữ	Ý nghĩa
Block / Cache Line	Đơn vị dữ liệu được nạp từ bộ nhớ chính vào Cache. Trong đề tài, mỗi block gồm 2 words (2 byte).
Set	Nhóm các Cache Line mà một block có thể được đặt vào. Với Direct-mapped, mỗi set chỉ có 1 line.
Tag	Phần bit cao của địa chỉ, được lưu cùng block để xác định block nào trong bộ nhớ chính đang nằm ở line đó.
Index	Phần bit dùng để chọn set (line) trong Cache.
Offset	Phần bit thấp dùng để chọn word (byte) bên trong block.
Valid Bit	Bit cho biết line đang chứa dữ liệu hợp lệ (1) hay chưa (0). Sau reset, mọi Valid Bit đều bằng 0.
Dirty Bit	Bit cho biết block đã bị sửa đổi trong Cache nhưng chưa ghi về bộ nhớ chính (chỉ dùng với Write-back).

2.3. Nguyên lý hoạt động của Cache

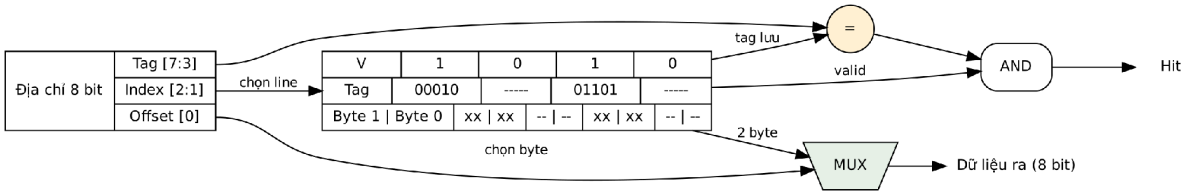
Khi CPU phát ra một địa chỉ, Cache chia địa chỉ đó thành ba trường Tag, Index và Offset. Số bit của mỗi trường được xác định theo công thức (2.1), (2.2) và (2.3):

$$\text{Số bit Offset} = \log_2(\text{Kích thước block tính theo đơn vị địa chỉ}) \qquad (2.1)$$

$$\text{Số bit Index} = \log_2(\text{Số set trong Cache}) \qquad (2.2)$$

$$\text{Số bit Tag} = \text{Độ rộng địa chỉ} - \text{Số bit Index} - \text{Số bit Offset} \qquad (2.3)$$

Áp dụng cho hệ thống của đề tài: địa chỉ rộng 8 bit (256 vị trí), mỗi block 2 byte nên Offset cần 1 bit, Cache có 4 set nên Index cần 2 bit, phần còn lại là Tag 5 bit. Như vậy địa chỉ được chia thành Tag = A[7:3], Index = A[2:1] và Offset = A[0].



Hình 2.2. Tổ chức và cơ chế kiểm tra của Cache Direct-mapped trong đề tài

Quá trình xử lý một yêu cầu đọc diễn ra như Hình 2.2: trường Index chọn ra một Cache Line; Tag lưu trong line được so sánh với Tag của địa chỉ; nếu Valid Bit bằng 1 và hai Tag trùng nhau thì yêu cầu là Cache Hit, khi đó trường Offset điều khiển bộ chọn (MUX) lấy đúng byte trong block để trả về CPU. Nếu không thỏa điều kiện, yêu cầu là Cache Miss và Cache phải lấy block tương ứng từ bộ nhớ chính, ghi vào line đã chọn, cập nhật Tag, đặt Valid Bit bằng 1 rồi mới trả dữ liệu về CPU.

2.4. Cache Hit và Cache Miss

Cache Hit xảy ra khi dữ liệu CPU yêu cầu đã có sẵn trong Cache; Cache Miss xảy ra khi dữ liệu chưa có và phải lấy từ mức bộ nhớ thấp hơn. Hiệu quả của Cache được đo bằng các đại lượng sau:

- **Hit Rate:** tỷ lệ số lần truy xuất Hit trên tổng số lần truy xuất, theo công thức (1.1).
- **Miss Rate:** tỷ lệ số lần Miss trên tổng số lần truy xuất, Miss Rate = 1 − Hit Rate.
- **Hit Time:** thời gian để truy xuất dữ liệu khi Hit, bao gồm thời gian kiểm tra Tag.
- **Miss Penalty:** thời gian thêm vào khi Miss, chủ yếu là thời gian nạp block từ bộ nhớ chính.

Thời gian truy xuất bộ nhớ trung bình (AMAT – Average Memory Access Time) được tính theo công thức (2.4):

$$AMAT = Hit\ Time + Miss\ Rate \times Miss\ Penalty \qquad (2.4)$$

Ví dụ, với Hit Time là 1 chu kỳ và Miss Penalty là 10 chu kỳ, nếu Hit Rate đạt 90% thì AMAT = 1 + 0,1 × 10 = 2 chu kỳ; nhưng nếu Hit Rate chỉ đạt 25% thì AMAT

= 1 + 0,75 × 10 = 8,5 chu kỳ. Điều này cho thấy Hit Rate ảnh hưởng rất lớn đến hiệu năng hệ thống. Các nguyên nhân gây ra Cache Miss thường được phân thành ba loại (mô hình 3C), trình bày trong Bảng 2.2.

Bảng 2.2. Phân loại Cache Miss theo mô hình 3C

Loại Miss	Đặc điểm
Compulsory (Cold) Miss	Xảy ra ở lần truy cập đầu tiên tới một block, khi block chưa từng được nạp vào Cache. Không thể tránh được, chỉ giảm bằng cách tăng kích thước block.
Capacity Miss	Xảy ra khi Cache không đủ dung lượng chứa tất cả các block chương trình đang dùng, buộc phải loại bỏ block rồi sau đó nạp lại.
Conflict Miss	Xảy ra khi nhiều block ánh xạ vào cùng một set và thay thế lẫn nhau dù Cache vẫn còn chỗ trống ở set khác. Đặc trưng của Direct-mapped và Set Associative.

2.5. Các phương pháp ánh xạ Cache

Phương pháp ánh xạ (Mapping) quy định một block của bộ nhớ chính được phép đặt vào những vị trí nào trong Cache. Có ba phương pháp chính:

- **Ánh xạ trực tiếp (Direct-mapped):** mỗi block chỉ được đặt vào đúng một line, xác định bởi Line = (Địa chỉ block) mod (Số line). Phần cứng đơn giản, chỉ cần một bộ so sánh Tag, thời gian Hit nhỏ, nhưng dễ xảy ra Conflict Miss.
- **Ánh xạ kết hợp toàn phần (Fully Associative):** một block có thể được đặt vào bất kỳ line nào. Không có Conflict Miss nhưng cần so sánh Tag với tất cả các line cùng lúc nên tốn nhiều phần cứng và năng lượng, chỉ phù hợp với Cache rất nhỏ.
- **Ánh xạ kết hợp tập hợp (N-way Set Associative):** Cache chia thành các set, mỗi set có N line (N way). Block được ánh xạ vào một set theo Set = (Địa chỉ block) mod (Số set) và có thể đặt vào bất kỳ way nào trong set đó. Đây là phương án dung hòa, được sử dụng phổ biến trong các bộ xử lý hiện nay.

Bảng 2.3. So sánh các phương pháp ánh xạ Cache

Tiêu chí	Direct-mapped	Set Associative	Fully Associative
Vị trí đặt block	1 line cố định	N line trong 1 set	Bất kỳ line nào
Số bộ so sánh Tag	1	N	Bằng số line
Conflict Miss	Nhiều	Ít hơn	Không có

Tiêu chí	Direct-mapped	Set Associative	Fully Associative
Chính sách thay thế	Không cần	Cần (LRU, FIFO…)	Cần (LRU, FIFO…)
Độ phức tạp phần cứng	Thấp	Trung bình	Cao

Đối với Set Associative và Fully Associative, khi set đã đầy, Cache cần một chính sách thay thế (Replacement Policy) để chọn block bị loại bỏ, phổ biến là LRU (Least Recently Used – loại block lâu nhất chưa được dùng), FIFO (First In First Out) và Random. Với Direct-mapped, vị trí đặt block là duy nhất nên block cũ tại line đó luôn bị thay thế mà không cần chính sách lựa chọn, đây là lý do đề tài lựa chọn kiến trúc này để đơn giản hóa thiết kế.

2.6. Các chính sách ghi dữ liệu

Khi CPU ghi dữ liệu, Cache cần đảm bảo tính nhất quán giữa dữ liệu trong Cache và trong bộ nhớ chính. Các chính sách được chia theo hai tình huống:

Khi ghi trúng Cache (Write Hit):

- **Write-through:** dữ liệu được ghi đồng thời vào Cache và bộ nhớ chính. Thiết kế đơn giản, bộ nhớ chính luôn được cập nhật, nhưng mỗi lần ghi đều phải truy cập bộ nhớ chính. Thường dùng kèm bộ đệm ghi (Write Buffer) để CPU không phải chờ.
- **Write-back:** dữ liệu chỉ được ghi vào Cache và đánh dấu Dirty Bit; block chỉ được ghi về bộ nhớ chính khi bị thay thế. Giảm đáng kể lưu lượng truy cập bộ nhớ nhưng điều khiển phức tạp hơn.

Khi ghi trượt Cache (Write Miss):

- **Write-allocate:** nạp block vào Cache rồi thực hiện ghi như Write Hit, thường đi kèm Write-back.
- **No-write-allocate:** chỉ ghi trực tiếp xuống bộ nhớ chính, không nạp block vào Cache, thường đi kèm Write-through.

Bảng 2.4. Các tổ hợp chính sách ghi phổ biến

Tiêu chí	Write-through + No-write-allocate	Write-back + Write-allocate
Tính nhất quán dữ liệu	Bộ nhớ chính luôn cập nhật	Bộ nhớ chính có thể cũ hơn Cache

Tiêu chí	Write-through + No-write-allocate	Write-back + Write-allocate
Lưu lượng ghi xuống bộ nhớ	Cao (mỗi lần ghi)	Thấp (chỉ khi thay thế block dirty)
Phần cứng bổ sung	Không cần Dirty Bit	Cần Dirty Bit, logic ghi ngược
Độ phức tạp điều khiển	Thấp	Cao

Đề tài lựa chọn tổ hợp Write-through và No-write-allocate vì logic điều khiển đơn giản, bộ nhớ chính luôn nhất quán với Cache và không cần bổ sung Dirty Bit, phù hợp với mục tiêu nghiên cứu nguyên lý của một hệ thống Cache quy mô nhỏ.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Về phạm vi và thông số thiết kế, đề tài tập trung phân tích và thiết kế mô hình hệ thống Cache Memory nhằm tối ưu hóa đường truyền dữ liệu giữa CPU và bộ nhớ chính. Hệ thống được xây dựng dựa trên kiến trúc Direct-mapped Cache. Bộ nhớ chính có dung lượng 256 words (dữ liệu 8 bit), bộ nhớ Cache gồm 4 sets (4 lines) và mỗi line lưu trữ một block gồm 2 words (2 byte). Địa chỉ truy cập của CPU có độ rộng 8 bit, được phân rã thành Tag 5 bit, Index 2 bit và Offset 1 bit. Hệ thống áp dụng chính sách ghi Write-through và No-write-allocate để giữ tính đồng nhất dữ liệu.

Bảng 3.1. Thông số thiết kế của hệ thống

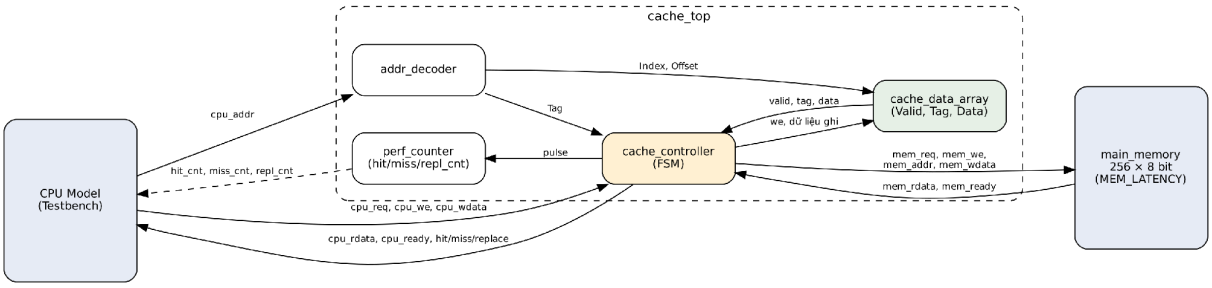
Thông số	Giá trị
Kiến trúc	Direct-mapped Cache
Dung lượng bộ nhớ chính	256 words × 8 bit
Số line của Cache	4 (4 sets × 1 way)
Kích thước block	2 words (2 byte)
Độ rộng địa chỉ	8 bit: Tag [7:3], Index [2:1], Offset [0]
Chính sách ghi	Write-through + No-write-allocate
Bộ đếm hiệu năng	Hit, Miss, Replacement (32 bit)

Về yêu cầu chức năng, hệ thống phải tiếp nhận yêu cầu từ CPU (qua các tín hiệu yêu cầu, địa chỉ, dữ liệu ghi và tín hiệu cho phép ghi), thực hiện giải mã địa chỉ và xác định trạng thái truy cập. Truy cập được coi là Cache Hit khi dòng Cache tương ứng có Valid Bit bằng 1 và giá trị Tag lưu trữ trùng khớp với Tag của địa chỉ. Khi xảy ra Cache Hit, dữ liệu được trả về cho CPU ngay trong chu kỳ hiện tại. Trường hợp Cache Miss, hệ thống phải phát lệnh đọc toàn bộ block 2 words từ bộ nhớ chính, cập nhật dữ liệu cùng Tag và Valid Bit vào Cache, sau đó mới hoàn tất giao dịch. Ngoài ra, hệ thống tích hợp khối đếm hiệu năng để ghi nhận số lần Hit, Miss, Replacement và hỗ trợ chức năng Reset đồng bộ toàn bộ trạng thái về ban đầu.

3.2. Kiến trúc tổng thể hệ thống

Hệ thống được chia thành ba tầng xử lý chính gồm khối CPU Model (Testbench tạo kích thích), khối hệ thống Cache trung gian (cache_top) và khối bộ nhớ chính (main_memory).

Bên trong cache_top, thiết kế tuân theo nguyên lý phân tách rõ ràng giữa đường dữ liệu (Datapath) và khối điều khiển (Control). Hệ thống được chia thành các module chức năng riêng biệt: module giải mã địa chỉ (addr_decoder), module mảng dữ liệu Cache (cache_data_array), bộ điều khiển FSM (cache_controller) và bộ đếm hiệu năng (perf_counter). Việc phân rã giúp quá trình đóng gói, mô phỏng kiểm thử độc lập cũng như mở rộng kiến trúc sau này trở nên thuận tiện. Sơ đồ khối của hệ thống được thể hiện ở Hình 3.1.



Hình 3.1. Sơ đồ khối tổng thể của hệ thống Cache Memory Simulator

3.3. Phân tích chức năng các module RTL

- **cache_top:** module cấu trúc đóng vai trò kết nối toàn bộ các thành phần con. Module tiếp nhận các tín hiệu từ phía CPU, chuyển tiếp đến bộ giải mã, bộ điều khiển và mảng dữ liệu, đồng thời liên kết với bộ nhớ chính và đưa các giá trị bộ đếm ra ngoài.
- **addr_decoder:** khối mạch tổ hợp nhận địa chỉ 8 bit từ CPU và tách thành ba trường: Tag (bit 7 đến 3), Index (bit 2 đến 1) và Offset (bit 0).
- **cache_controller:** bộ điều khiển trung tâm sử dụng máy trạng thái hữu hạn (FSM), chuyển đổi giữa các trạng thái chờ yêu cầu (IDLE), kiểm tra so sánh Tag (LOOKUP), nạp từng word từ bộ nhớ chính (REFILL_W0, REFILL_W1) và phản hồi kết quả (RESPOND). Khối này điều khiển tín hiệu ghi mảng Cache, phát yêu cầu tới bộ nhớ chính và tạo các xung báo hiệu Hit, Miss, Replacement.
- **cache_data_array:** lưu trữ Valid Bit, Tag và dữ liệu cho 4 dòng Cache. Khối thực hiện đọc tổ hợp để phục vụ so sánh Hit/Miss ngay trong chu kỳ kiểm tra và ghi dữ liệu mới theo tín hiệu điều khiển từ Controller khi kết thúc quá trình nạp.

- **main_memory:** mô phỏng bộ nhớ chính dung lượng 256 words với độ trễ phản hồi cố định (MEM_LATENCY). Khi nhận yêu cầu đọc, khối đếm đủ số chu kỳ trễ rồi xuất dữ liệu và bật tín hiệu mem_ready.
- **perf_counter:** gồm các bộ đếm 32 bit hoạt động đồng bộ, tăng giá trị mỗi khi nhận xung Hit, Miss hoặc Replacement từ Controller, phục vụ việc tính Hit Rate.

3.4. Mô tả tín hiệu giao tiếp hệ thống

Hệ thống sử dụng xung clock chính clk và tín hiệu reset tích cực mức thấp rst_n. Các tín hiệu giao tiếp phía CPU và phía bộ nhớ chính được mô tả trong Bảng 3.2 và Bảng 3.3.

Bảng 3.2. Tín hiệu giao tiếp phía CPU

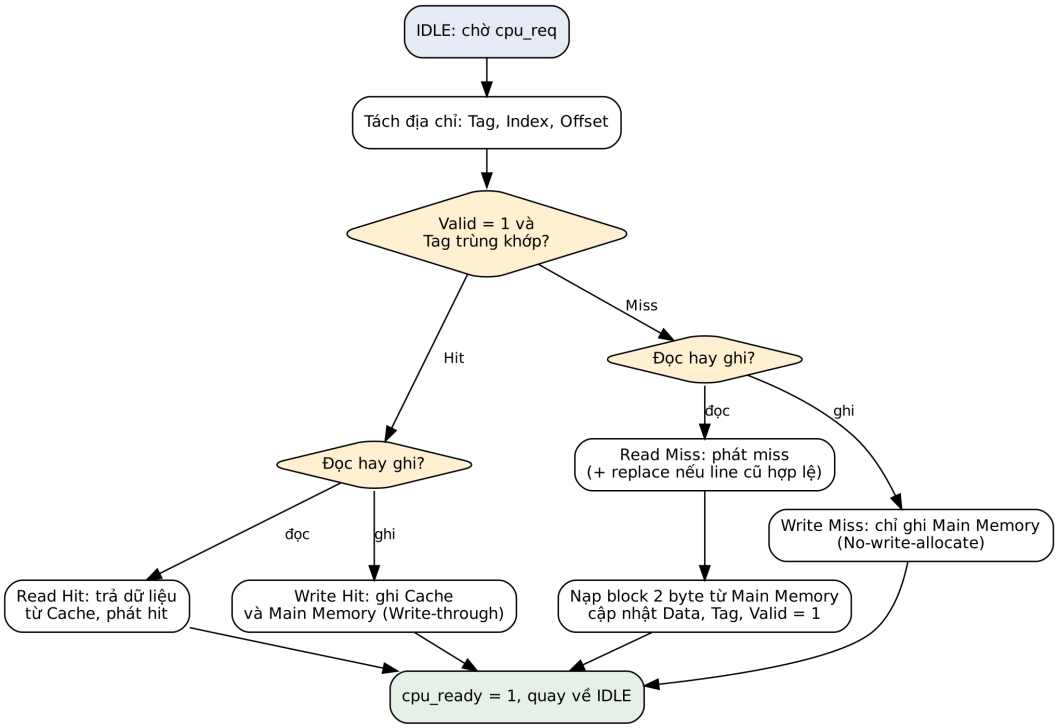
Tín hiệu	Độ rộng	Chiều	Chức năng
clk	1	Input	Xung clock của hệ thống
rst_n	1	Input	Reset đồng bộ, tích cực mức thấp
cpu_req	1	Input	Yêu cầu truy cập từ CPU
cpu_we	1	Input	Chọn chế độ: 1 = ghi, 0 = đọc
cpu_addr	8	Input	Địa chỉ truy cập
cpu_wdata	8	Input	Dữ liệu ghi
cpu_rdata	8	Output	Dữ liệu đọc trả về CPU
cpu_ready	1	Output	Báo giao dịch đã hoàn tất
hit / miss / replace	1	Output	Xung báo trạng thái, rộng một chu kỳ
hit_cnt / miss_cnt / repl_cnt	32	Output	Giá trị các bộ đếm hiệu năng

Bảng 3.3. Tín hiệu giao tiếp phía bộ nhớ chính

Tín hiệu	Độ rộng	Chiều	Chức năng
mem_req	1	Output	Yêu cầu truy cập bộ nhớ chính
mem_we	1	Output	Chọn chế độ: 1 = ghi, 0 = đọc
mem_addr	8	Output	Địa chỉ gửi tới bộ nhớ chính
mem_wdata	8	Output	Dữ liệu ghi xuống bộ nhớ chính
mem_rdata	8	Input	Dữ liệu đọc từ bộ nhớ chính
mem_ready	1	Input	Báo dữ liệu đã sẵn sàng

3.5. Luồng dữ liệu và thực thi hệ thống

Luồng xử lý bắt đầu khi CPU phát tín hiệu yêu cầu truy cập kèm địa chỉ. Khối giải mã địa chỉ lập tức tách địa chỉ thành Tag, Index và Offset. Lưu đồ xử lý tổng quát được thể hiện ở Hình 3.2.



Hình 3.2. Lưu đồ xử lý yêu cầu đọc/ghi của hệ thống

Ở trạng thái LOOKUP, cache_data_array cung cấp thông tin của dòng Cache tương ứng với Index. Nếu Valid Bit bằng 1 và Tag lưu trữ trùng với Tag của địa chỉ, Controller xác định Read Hit, kích hoạt xung hit, chuyển sang RESPOND, trả dữ liệu về CPU cùng tín hiệu cpu_ready và quay về IDLE mà không cần truy cập bộ nhớ chính.

Nếu Tag không trùng hoặc Valid Bit bằng 0, Controller xác định Read Miss, phát xung miss (đồng thời phát xung replace nếu dòng Cache cũ đang chứa dữ liệu hợp lệ). Sau đó, Controller lần lượt chuyển qua REFILL_W0 và REFILL_W1 để đọc từng word của block từ bộ nhớ chính. Khi nhận đủ dữ liệu, Controller ghi Tag mới, bật Valid Bit, cập nhật dữ liệu vào Cache rồi chuyển sang RESPOND để trả dữ liệu cho CPU. Đối với thao tác ghi, hệ thống áp dụng Write-through và No-write-allocate như đã trình bày ở Chương 2.

Bảng 3.4 minh họa luồng xử lý với chuỗi truy xuất mẫu 0x10, 0x11, 0x20, 0x10 khi Cache ban đầu trống.

Bảng 3.4. Ví dụ thực thi chuỗi truy xuất mẫu

Lần	Truy cập	Tag / Index / Offset	Kết quả	Giải thích
1	READ 0x10	00010 / 00 / 0	MISS	Cold miss, nạp block 0x10–0x11 vào line 0
2	READ 0x11	00010 / 00 / 1	HIT	Cùng block với 0x10 (cục bộ không gian)
3	READ 0x20	00100 / 00 / 0	MISS + Replace	Cùng Index nhưng khác Tag, block 0x20–0x21 ghi đè line 0
4	READ 0x10	00010 / 00 / 0	MISS + Replace	Block 0x10 đã bị thay thế (conflict miss)

CHƯƠNG 4. TRIỂN KHAI THIẾT KẾ RTL

4.1. Môi trường và công cụ thiết kế

Nhóm sử dụng ngôn ngữ mô tả phần cứng Verilog HDL để xây dựng các thành phần của hệ thống Cache Memory. Công cụ chính là ModelSim Intel FPGA Starter Edition 2020.1, dùng để biên dịch, mô phỏng và quan sát dạng sóng tín hiệu.

Cấu hình hệ thống gồm Cache Direct-mapped 4 line, mỗi line chứa 2 byte dữ liệu, bộ nhớ chính 256 byte và địa chỉ 8 bit được chia thành Tag 5 bit, Index 2 bit và Offset 1 bit.

4.2. Thiết kế module Cache Controller

Cache Controller là bộ điều khiển trung tâm, có nhiệm vụ nhận yêu cầu từ CPU, kiểm tra dữ liệu trong Cache và điều khiển quá trình đọc hoặc ghi dữ liệu. Controller được hiện thực bằng máy trạng thái hữu hạn (FSM) gồm 6 trạng thái, mô tả trong Bảng 4.1.

Bảng 4.1. Các trạng thái của FSM trong Cache Controller

Trạng thái	Chức năng
IDLE	Chờ yêu cầu từ CPU
CHECK	Kiểm tra Valid Bit và Tag để xác định Cache Hit hoặc Cache Miss
MISS	Gửi yêu cầu đọc dữ liệu tới Main Memory
REFILL	Nạp block dữ liệu từ Main Memory vào Cache, cập nhật Tag và Valid Bit
WRITE_WAIT	Chờ Main Memory hoàn thành thao tác ghi (Write-through)
DONE	Thông báo giao dịch đã hoàn thành (cpu_ready = 1)

Việc sử dụng FSM giúp các thao tác được thực hiện đúng trình tự, tách biệt rõ các giai đoạn kiểm tra, truy cập bộ nhớ và phản hồi, đảm bảo hệ thống hoạt động ổn định.

4.3. Thiết kế module Cache Data Array

4.3.1. Chức năng của module

Module cache_data_array là khối lưu trữ dữ liệu của hệ thống, chứa dữ liệu của các Cache Line, cho phép Cache Controller ghi dữ liệu mới vào line được chọn bằng Index và đọc lại dữ liệu của line đó. Module không so sánh Tag và không xác định

Hit/Miss; việc lưu Tag, Valid Bit và kiểm tra Hit/Miss thuộc về khối quản lý Tag/Valid và Cache Controller.

Theo đặc tả hệ thống, Cache có 4 line, mỗi line chứa 2 byte (tổng cộng 8 byte dữ liệu). Module cần hỗ trợ hai cách ghi:

- **Block Write:** ghi toàn bộ 2 byte vào Cache khi nạp block từ Main Memory sau Cache Miss.
- **Word Write:** ghi một byte khi CPU thực hiện thao tác ghi trúng Cache.

Khi CPU đọc dữ liệu, hệ thống sử dụng Index để chọn line và Offset để chọn byte cần truy xuất. Trong giai đoạn hiện tại, nhóm đã hiện thực và kiểm chứng phiên bản cơ sở của module với mỗi line lưu 1 byte, làm nền tảng để mở rộng lên 2 byte. Thông số của phiên bản này được trình bày trong Bảng 4.2.

Bảng 4.2. Thông số thiết kế phiên bản hiện tại của Cache Data Array

Thông số	Giá trị
Số Cache Line	4
Độ rộng dữ liệu mỗi line	8 bit (1 byte)
Độ rộng Index	2 bit
Cách ghi	Đồng bộ theo cạnh lên của clock, khi data_we = 1
Cách đọc	Tổ hợp (combinational) theo data_index
Reset	Đồng bộ, tích cực mức cao (rst), xóa cả 4 line về 0

4.3.2. Giao tiếp tín hiệu

Bảng 4.3. Tín hiệu giao tiếp của module cache_data_array

Tín hiệu	Độ rộng	Chiều	Chức năng
clk	1 bit	Input	Xung clock đồng bộ
rst	1 bit	Input	Reset, tích cực mức cao, xóa dữ liệu 4 line về 0
data_index	2 bit	Input	Chọn 1 trong 4 Cache Line
cache_data_in	8 bit	Input	Dữ liệu cần ghi vào Cache (khi CPU ghi hoặc khi nạp từ Main Memory)
data_we	1 bit	Input	Cho phép ghi vào Cache Data Array
cache_data_out	8 bit	Output	Dữ liệu đọc ra từ line được chọn

4.3.3. Nguyên lý hoạt động

- **Reset:** khi rst bằng 1, tại cạnh lên của clock, toàn bộ 4 Cache Line được đưa về 0. Trước lần reset đầu tiên, nội dung bộ nhớ chưa xác định nên đầu ra mang giá

trị x trong mô phỏng; việc xóa về 0 giúp tránh giá trị x lan sang các module khác.

- **Ghi dữ liệu:** khi rst bằng 0 và data_we bằng 1, tại cạnh lên của clock, giá trị cache_data_in được ghi vào line có chỉ số data_index; các line còn lại giữ nguyên. Khi data_we bằng 0, nội dung các line không thay đổi dù cache_data_in thay đổi.
- **Đọc dữ liệu:** cache_data_out luôn lấy giá trị của line có chỉ số data_index. Vì việc đọc là tổ hợp nên dữ liệu có ngay trong chu kỳ hiện tại, phù hợp với yêu cầu trả dữ liệu về CPU ngay khi xảy ra Cache Hit.

4.3.4. Mã nguồn Verilog

```
module cache_data_array (
    input  wire       clk,
    input  wire       rst,
    input  wire [1:0] data_index,     // chon 1 trong 4 Cache Lines
    input  wire [7:0] cache_data_in,  // du lieu ghi
    input  wire       data_we,        // cho phep ghi
    output wire [7:0] cache_data_out  // du lieu doc tu Cache
);
    reg [7:0] data_mem [0:3];
    integer i;

    // Ghi dong bo / reset xoa toan bo data ve 0
    always @(posedge clk) begin
        if (rst) begin
            for (i = 0; i < 4; i = i + 1)
                data_mem[i] <= 8'd0;
        end
        else if (data_we) begin
            data_mem[data_index] <= cache_data_in;
        end
    end

    // Doc to hop
    assign cache_data_out = data_mem[data_index];
endmodule
```

4.4. Thiết kế các module hỗ trợ

Ngoài hai module chính, hệ thống có các module hỗ trợ sau:

- **Cache Tag Array:** lưu trữ Tag (5 bit) và Valid Bit của từng Cache Line. Valid Bit bằng 0 cho biết line chưa chứa dữ liệu hợp lệ, bằng 1 cho biết dữ liệu hợp lệ. Khi reset, toàn bộ Valid Bit được xóa về 0.
- **Main Memory:** bộ nhớ chính dung lượng 256 byte, mỗi địa chỉ lưu 1 byte. Module cung cấp dữ liệu cho Cache khi xảy ra Cache Miss và tiếp nhận dữ liệu khi CPU thực hiện thao tác ghi, có độ trễ phản hồi cố định MEM_LATENCY.

- **Address Decoder và Performance Counter:** bộ giải mã tách địa chỉ thành Tag, Index, Offset; bộ đếm hiệu năng ghi nhận số lần Hit, Miss và Replacement như đã phân tích ở Chương 3.

4.5. Kết nối các module tại Cache Top

Module cache_top kết nối toàn bộ hệ thống gồm Cache Controller, Cache Tag Array, Cache Data Array, Main Memory và các khối hỗ trợ theo sơ đồ ở Hình 3.1. Khi CPU gửi yêu cầu, cache_top tiếp nhận địa chỉ và chia thành Tag, Index và Offset. Sau đó, Cache Controller phối hợp với các module còn lại để kiểm tra Cache, đọc dữ liệu hoặc truy cập Main Memory khi cần thiết. Việc chia hệ thống thành các module riêng biệt giúp mã nguồn dễ quản lý, kiểm tra và sửa lỗi.

4.6. Nguyên lý hoạt động của hệ thống

Hệ thống có hai trường hợp truy cập chính là Cache Hit và Cache Miss.

- **Cache Hit:** khi CPU yêu cầu đọc dữ liệu, Cache Controller kiểm tra Tag và Valid Bit. Nếu Valid Bit bằng 1 và Tag trùng khớp với địa chỉ yêu cầu, dữ liệu được lấy trực tiếp từ Cache và trả về CPU mà không cần truy cập Main Memory.
- **Cache Miss:** nếu dữ liệu chưa tồn tại trong Cache hoặc Tag không trùng khớp, Cache Controller gửi yêu cầu đến Main Memory. Sau khi nhận được dữ liệu, hệ thống nạp 2 byte vào Cache, cập nhật Tag và Valid Bit, sau đó trả dữ liệu về CPU.

Đối với thao tác ghi, hệ thống sử dụng chính sách Write-through kết hợp No-write-allocate: khi ghi trúng Cache, dữ liệu được cập nhật cả trong Cache và Main Memory; khi ghi trượt Cache, dữ liệu chỉ được ghi xuống Main Memory mà không nạp block vào Cache.

4.7. Kết luận chương

Chương 4 đã trình bày quá trình triển khai thiết kế RTL cho hệ thống Cache Memory Simulator. Thông qua việc xây dựng và kết nối các module bằng Verilog HDL, nhóm đã mô tả được quá trình đọc, ghi dữ liệu cũng như cách hệ thống xử lý Cache Hit và Cache Miss, làm cơ sở cho quá trình mô phỏng và kiểm chứng ở Chương 5.

CHƯƠNG 5. MÔ PHỎNG VÀ KIỂM CHỨNG

5.1. Phương pháp kiểm chứng

Sau khi hoàn thành thiết kế, nhóm tiến hành mô phỏng và kiểm chứng bằng phần mềm ModelSim. Testbench được sử dụng để tạo các tín hiệu đầu vào, thực hiện các thao tác đọc, ghi và Reset, đồng thời tự động so sánh kết quả đầu ra với giá trị mong đợi. Mỗi phép kiểm tra in ra PASS nếu kết quả đúng; ngược lại in FAIL kèm giá trị mong đợi, giá trị thực tế và dừng mô phỏng.

Quá trình kiểm chứng tập trung vào các nội dung:

- Kiểm tra hoạt động của Clock và Reset.
- Kiểm tra chức năng đọc và ghi dữ liệu.
- Kiểm tra trạng thái hợp lệ của Cache Line.
- Kiểm tra Cache Hit và Cache Miss.
- Kiểm tra chức năng ghi đè dữ liệu.
- Kiểm tra sự phối hợp giữa các module trong hệ thống.

5.2. Các trường hợp kiểm thử

Nhóm xây dựng các trường hợp kiểm thử nhằm đánh giá chức năng của từng module và hoạt động tổng thể của hệ thống, được liệt kê trong Bảng 5.1.

Bảng 5.1. Các trường hợp kiểm thử của hệ thống

STT	Trường hợp kiểm thử	Mục đích
1	Initial Reset	Kiểm tra khởi tạo hệ thống
2	Write Operation	Kiểm tra chức năng ghi dữ liệu
3	Read Operation	Kiểm tra chức năng đọc dữ liệu
4	Cache Hit	Kiểm tra truy cập dữ liệu đã tồn tại trong Cache
5	Cache Miss	Kiểm tra truy cập dữ liệu chưa tồn tại trong Cache
6	Invalid Line Detection	Kiểm tra phát hiện Cache Line không hợp lệ
7	Overwrite Operation	Kiểm tra ghi đè dữ liệu
8	Reset Verification	Kiểm tra Reset sau khi ghi dữ liệu

Đối với module cache_data_array, testbench tb_cache_data_array tạo clock chu kỳ 10 ns, điều khiển các tín hiệu rst, data_index, cache_data_in, data_we và kiểm tra

cache_data_out bằng task check_output. Các trường hợp kiểm thử được trình bày trong Bảng 5.2.

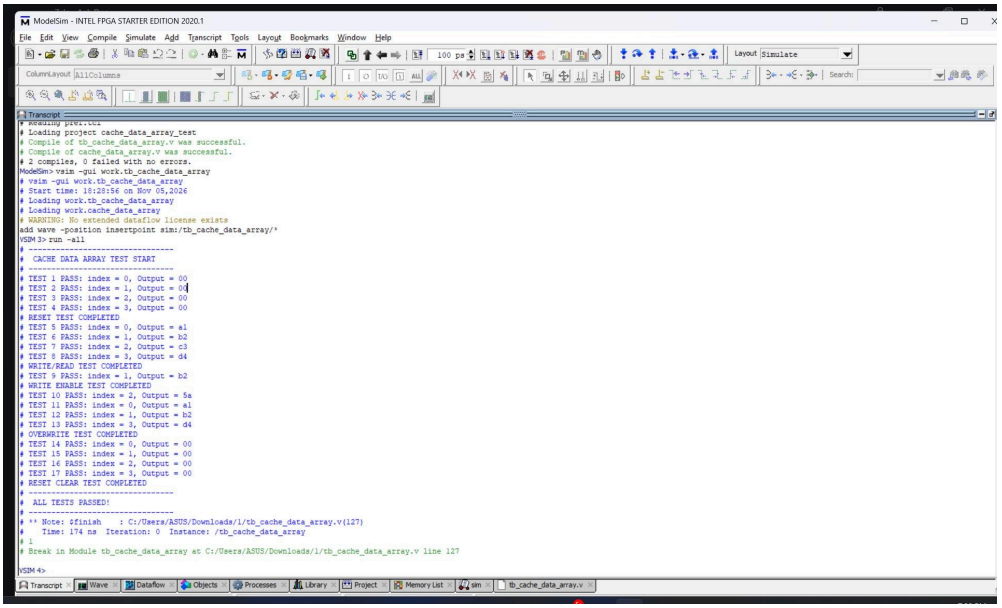
Bảng 5.2. Các trường hợp kiểm thử module Cache Data Array

STT	Trường hợp	Mục đích	Test trong log
1	Initial Reset	Sau reset, cả 4 line đọc ra 00	TEST 1–4
2	Write/Read	Ghi A1, B2, C3, D4 vào line 0 đến 3 rồi đọc lại	TEST 5–8
3	Write Enable	Khi data_we = 0 thì dữ liệu không bị thay đổi	TEST 9
4	Overwrite	Ghi đè 5A vào line 2, các line 0, 1, 3 giữ nguyên	TEST 10–13
5	Reset Clear	Reset giữa chừng xóa cả 4 line về 00	TEST 14–17

5.3. Kết quả mô phỏng

5.3.1. Module Cache Data Array

Hai file cache_data_array.v và tb_cache_data_array.v biên dịch thành công trên ModelSim (2 compiles, 0 failed with no errors). Kết quả trong cửa sổ Transcript cho thấy cả 17 phép kiểm tra đều PASS và testbench kết thúc bằng thông báo “ALL TESTS PASSED!”. Mô phỏng dừng bằng lệnh $finish tại thời điểm 174 ns.



Hình 5.1. Kết quả mô phỏng module Cache Data Array trong cửa sổ Transcript

5.3.2. Khối lưu trữ Cache (Cache Storage)

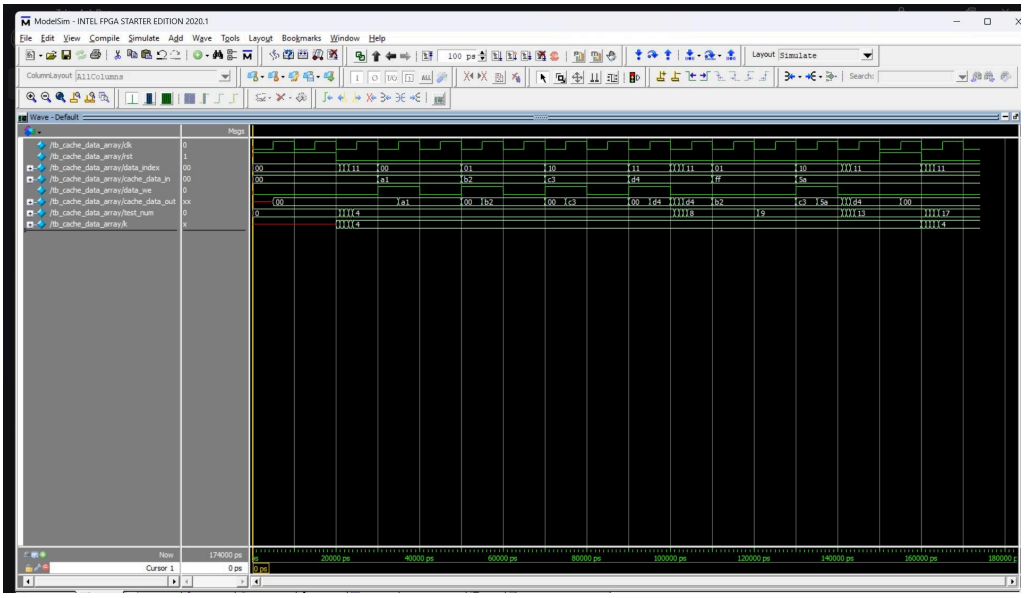
Đối với khối lưu trữ Cache, kết quả mô phỏng ghi nhận các thông báo: TEST 1 PASS (Initial Reset), TEST 2 PASS (Invalid Line Detection), PASS cho Cache Line 0, 1, 2, 3, PASS cho Cache Line 2 sau khi ghi đè, "Overwrite completed", "All cache lines reset" và "ADVANCED SIMULATION COMPLETED". Kết quả cho thấy các Cache Line lưu trữ dữ liệu, ghi đè và khôi phục trạng thái sau Reset đúng theo thiết kế.

Đối với toàn bộ hệ thống, kết quả kiểm chứng cần được đánh giá thêm thông qua quá trình tích hợp và mô phỏng Cache Top để xác nhận sự phối hợp giữa Cache Controller, Cache Data Array và các module hỗ trợ.

5.4. Phân tích dạng sóng tín hiệu (Waveform)

Waveform trong ModelSim được sử dụng để quan sát sự thay đổi của các tín hiệu theo thời gian mô phỏng. Các tín hiệu quan trọng gồm: Clock, Reset, Read Enable, Write Enable, Address, Write Data, Read Data, Valid và Hit/Miss. Thông qua Waveform, nhóm kiểm tra trình tự hoạt động của các tín hiệu, xác định thời điểm đọc và ghi dữ liệu và đánh giá tính chính xác của quá trình điều khiển.

Đối với module Cache Data Array, tín hiệu rst ở mức 1 trong hai chu kỳ đầu rồi về 0. Đầu ra cache_data_out ban đầu có giá trị không xác định (x), sau cạnh lên clock đầu tiên khi đang reset thì chuyển về 00. Tiếp theo, testbench lần lượt ghi các giá trị a1, b2, c3, d4 vào line 0 đến 3 (data_we lên 1 trong mỗi lần ghi) và cache_data_out trả về đúng giá trị tại line được chọn. Ở bước kiểm tra Write Enable, cache_data_in đổi sang ff nhưng data_we bằng 0 nên line 1 vẫn giữ b2. Sau khi ghi đè 5a vào line 2, các line 0, 1, 3 vẫn giữ a1, b2, d4. Cuối cùng, reset lần hai đưa cả 4 line về 00.



Hình 5.3. Dạng sóng tín hiệu của module Cache Data Array

5.5. Đánh giá kết quả

5.5.1. Kết quả đạt được

- Xây dựng được cấu trúc hệ thống gồm Cache Controller, Cache Data Array và các module hỗ trợ.
- Module Cache Data Array đạt 17/17 phép kiểm tra; khối Cache Storage đạt toàn bộ các trường hợp đã kiểm tra.
- Kiểm tra được chức năng khởi tạo, Reset, đọc, ghi, ghi đè và trạng thái hợp lệ của các Cache Line.
- Sử dụng ModelSim để quan sát và phân tích hoạt động của thiết kế qua Transcript và Waveform.

5.5.2. Hạn chế

- Quá trình kiểm chứng chủ yếu thực hiện ở mức module, chưa mô phỏng tích hợp toàn hệ thống.
- Số lượng trường hợp kiểm thử chưa bao phủ tất cả tình huống thực tế.
- Chưa đánh giá đầy đủ hiệu năng (Hit Rate đo từ bộ đếm) và chưa kiểm chứng trên phần cứng thực tế.

5.5.3. Kết luận chương

Chương 5 đã trình bày quá trình mô phỏng và kiểm chứng hệ thống Cache Memory Simulator RTL bằng ModelSim. Thông qua Testbench, Transcript và Waveform, nhóm đã kiểm tra các chức năng quan trọng gồm đọc, ghi, Reset và quản lý trạng thái Cache Line. Các trường hợp kiểm thử đã thực hiện đều đạt yêu cầu, là cơ sở để đánh giá thiết kế và tiếp tục hoàn thiện hệ thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết quá trình thực hiện đề tài "Cache Memory Simulator RTL", đối chiếu kết quả đạt được với các mục tiêu đã đặt ra tại Chương 1, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo.

6.1. Những kết quả đạt được

6.1.1. Về mặt lý thuyết

Nhóm đã hệ thống hóa được các kiến thức nền tảng về tổ chức bộ nhớ: vai trò của Cache giữa CPU và bộ nhớ chính, nguyên lý cục bộ, các khái niệm Cache Hit, Cache Miss, Hit Rate, AMAT, mô hình 3C, các phương pháp ánh xạ (Direct-mapped, Set Associative, Fully Associative) và các chính sách ghi (Write-through, Write-back, Write-allocate, No-write-allocate). Đây là cơ sở để nhóm lựa chọn cấu hình và đưa ra các quyết định thiết kế.

6.1.2. Về mặt thiết kế hệ thống

Nhóm đã xác định đầy đủ đặc tả cho một hệ thống Cache quy mô nhỏ:

- Kiến trúc Direct-mapped gồm 4 sets (4 Cache Line), mỗi line chứa một block 2 words (2 byte).
- Bộ nhớ chính 256 byte, địa chỉ 8 bit được chia thành Tag 5 bit, Index 2 bit và Offset 1 bit.
- Chính sách ghi Write-through kết hợp No-write-allocate đảm bảo tính nhất quán dữ liệu.

Hệ thống được phân rã theo nguyên tắc tách biệt đường dữ liệu và khối điều khiển, gồm Cache Controller (FSM), Cache Data Array, khối Tag/Valid, Main Memory có độ trễ cố định, bộ giải mã địa chỉ và bộ đếm hiệu năng. Các tín hiệu giao tiếp và luồng xử lý cho Read Hit, Read Miss, Write Hit, Write Miss đã được mô tả rõ ràng bằng sơ đồ khối và lưu đồ.

6.1.3. Về mặt hiện thực và kiểm chứng

Các module được mô tả bằng Verilog HDL và mô phỏng trên ModelSim Intel FPGA Starter Edition 2020.1:

- **Module Cache Data Array:** testbench tự kiểm tra gồm 17 phép kiểm tra thuộc 5 nhóm (Initial Reset, Write/Read, Write Enable, Overwrite, Reset Clear). Thiết

kế biên dịch không lỗi, cả 17 phép kiểm tra đều PASS, mô phỏng kết thúc với thông báo "ALL TESTS PASSED!" tại 174 ns.

- **Khối Cache Storage:** các trường hợp Initial Reset, Invalid Line Detection, ghi/đọc trên cả 4 Cache Line, ghi đè và Reset sau khi ghi đều cho kết quả đúng.
- **Phân tích dạng sóng:** Waveform xác nhận dữ liệu chỉ được ghi khi tín hiệu cho phép ghi ở mức 1, việc đọc là tổ hợp nên dữ liệu có ngay trong chu kỳ hiện tại, và reset đưa toàn bộ dữ liệu về 0, tránh lan truyền giá trị không xác định.

Bên cạnh đó, phân tích theo thiết kế với chuỗi truy xuất mẫu 0x10, 0x11, 0x20, 0x10 cho kết quả 1 Hit, 3 Miss (1 cold miss, 2 conflict miss) và 2 lần thay thế, tương ứng Hit Rate = 1/4 = 25% (Bảng 6.1). Kết quả minh họa rõ lợi ích của cục bộ không gian (lần truy xuất 0x11) và nhược điểm xung đột ánh xạ của Direct-mapped.

Bảng 6.1. Kết quả phân tích chuỗi truy xuất mẫu trên Cache Direct-mapped

Lần	Địa chỉ	Index	Kết quả	Giải thích
1	READ 0x10	00	MISS	Cache trống (cold miss), nạp block 0x10–0x11 vào line 0
2	READ 0x11	00	HIT	Cùng block với 0x10 (cục bộ không gian)
3	READ 0x20	00	MISS + Replace	Cùng Index, khác Tag, block 0x20–0x21 ghi đè line 0
4	READ 0x10	00	MISS + Replace	Block 0x10 đã bị thay thế (conflict miss)

Bảng 6.2 tổng hợp mức độ hoàn thành của đề tài so với các mục tiêu cụ thể đã đặt ra tại Chương 1.

Bảng 6.2. Đối chiếu kết quả đạt được với mục tiêu đề tài

STT	Mục tiêu (Chương 1)	Mức độ	Ghi chú
1	Nghiên cứu cơ sở lý thuyết về Cache Memory	Hoàn thành	Trình bày tại Chương 2
2	Xây dựng kiến trúc Cache Direct-mapped 4 sets, block 2 words	Hoàn thành	Tag 5 bit, Index 2 bit, Offset 1 bit
3	Thiết kế các khối chức năng RTL	Hoàn thành ở mức module	Cache Data Array cần mở rộng lên 2 byte/line
4	Xây dựng môi trường kiểm thử	Hoàn thành ở mức module	Testbench tự kiểm tra PASS/FAIL

STT	Mục tiêu (Chương 1)	Mức độ	Ghi chú
5	Xác định Cache Hit/Miss và tính Hit Rate	Đã phân tích theo thiết kế	Chưa có số liệu từ mô phỏng Cache Top
6	Kiểm tra cơ chế thay thế dữ liệu	Đã phân tích, kiểm tra ghi đè ở mức module	Cần xác nhận khi tích hợp toàn hệ thống
7	Đánh giá hệ thống, đề xuất hướng phát triển	Hoàn thành	Trình bày tại Chương 6

6.1.4. Về mặt kỹ năng

Quá trình thực hiện giúp các thành viên rèn luyện quy trình thiết kế phần cứng số từ đặc tả, phân tích kiến trúc, thiết kế RTL đến viết testbench và gỡ lỗi bằng waveform; đồng thời nâng cao kỹ năng sử dụng ModelSim, làm việc nhóm, phân chia module và viết báo cáo kỹ thuật.

6.2. Những hạn chế

- **Chưa kiểm chứng toàn hệ thống:** các module mới được mô phỏng độc lập; Cache Top chưa được mô phỏng tích hợp nên Hit, Miss, Replacement và Hit Rate mới dừng ở mức phân tích, chưa có số liệu đo từ bộ đếm hiệu năng.
- **Module chưa hoàn chỉnh theo đặc tả:** phiên bản Cache Data Array đã kiểm chứng mới lưu 1 byte trên mỗi line, chưa hỗ trợ Block Write, Word Write và chọn byte theo Offset.
- **Quy ước giao tiếp chưa thống nhất:** một số quy ước giữa các module như mức tích cực của tín hiệu reset và tập trạng thái của FSM chưa được đồng bộ hoàn toàn, cần chuẩn hóa trước khi tích hợp.
- **Giới hạn của kiến trúc:** Direct-mapped đơn giản nhưng dễ xảy ra conflict miss khi nhiều block ánh xạ vào cùng một set; Write-through làm tăng số lần truy cập bộ nhớ chính khi ghi.
- **Quy mô và độ bao phủ kiểm thử:** Cache chỉ có 4 line, dữ liệu 8 bit; kiểm thử là kiểm thử định hướng (directed test) với số lượng nhỏ, chưa có kiểm thử ngẫu nhiên, mô hình tham chiếu hay đo độ bao phủ.
- **Chưa triển khai trên phần cứng:** thiết kế chưa được tổng hợp và nạp lên FPGA nên chưa đánh giá được tần số hoạt động tối đa, tài nguyên sử dụng và công suất tiêu thụ.

6.3. Hướng phát triển trong tương lai

6.3.1. Hoàn thiện và tích hợp hệ thống

- Mở rộng Cache Data Array lên 2 byte mỗi line, bổ sung Block Write, Word Write và chọn byte theo Offset.
- Thống nhất quy ước reset, tên tín hiệu và tập trạng thái FSM giữa các module, sau đó mô phỏng tích hợp Cache Top với chuỗi truy xuất mẫu để xác nhận kết quả 1 Hit / 3 Miss / 2 Replace và Hit Rate 25% bằng bộ đếm hiệu năng.
- Bổ sung kiểm thử cho thao tác ghi (Write Hit, Write Miss) để xác nhận chính sách Write-through và No-write-allocate.

6.3.2. Mở rộng kiến trúc Cache

- Hiện thực Cache 2-way Set Associative với chính sách thay thế LRU và so sánh với Direct-mapped trên cùng dung lượng. Theo phân tích, với chuỗi truy xuất mẫu, Hit Rate có thể tăng từ 25% lên 50% (Bảng 6.3).
- Hỗ trợ Write-back kết hợp Write-allocate với Dirty Bit nhằm giảm số lần ghi xuống bộ nhớ chính, và so sánh lưu lượng truy cập bộ nhớ giữa hai chính sách.
- Tham số hóa thiết kế (số line, kích thước block, độ kết hợp, độ rộng dữ liệu) bằng parameter trong Verilog để dễ dàng thử nghiệm nhiều cấu hình.

Bảng 6.3. So sánh dự kiến giữa Direct-mapped và 2-way Set Associative

Tiêu chí	Direct-mapped (hiện tại)	2-way Set Associative (đề xuất)
Tổ chức	4 sets × 1 way	2 sets × 2 ways (cùng dung lượng 4 line)
Phân chia địa chỉ	Tag 5 bit, Index 2 bit, Offset 1 bit	Tag 6 bit, Index 1 bit, Offset 1 bit
Chuỗi 0x10, 0x11, 0x20, 0x10	MISS, HIT, MISS, MISS	MISS, HIT, MISS, HIT
Hit Rate (chuỗi ví dụ)	1/4 = 25%	2/4 = 50%
Chi phí phần cứng	1 bộ so sánh Tag, không cần chính sách thay thế	2 bộ so sánh Tag, thêm bit LRU và bộ chọn way

6.3.3. Nâng cao phương pháp kiểm chứng

- Xây dựng testbench tự kiểm tra với mô hình tham chiếu (reference model) và chuỗi địa chỉ ngẫu nhiên có ràng buộc.

- Chuyển sang SystemVerilog, sử dụng assertion và functional coverage, hướng tới môi trường kiểm chứng theo phương pháp UVM.

6.3.4. Triển khai trên phần cứng và ứng dụng

- Tổng hợp thiết kế bằng Intel Quartus Prime và nạp lên FPGA để đánh giá tần số hoạt động, tài nguyên và công suất.
- Kết nối Cache với một lõi CPU đơn giản (ví dụ RISC-V) để chạy chương trình thực tế; nghiên cứu Cache nhiều mức (L1/L2) và tách riêng Instruction Cache, Data Cache.

6.4. Kết luận chung

Đề tài đã xây dựng được kiến trúc và các khối chức năng chính của một hệ thống Cache Memory Direct-mapped quy mô nhỏ bằng Verilog HDL, kiểm chứng thành công ở mức module trên ModelSim và phân tích được hành vi Hit, Miss, thay thế dữ liệu của hệ thống. Mặc dù việc mô phỏng tích hợp toàn hệ thống và triển khai trên phần cứng vẫn cần được tiếp tục, kết quả hiện tại là nền tảng vững chắc để nhóm hoàn thiện thiết kế, mở rộng sang các kiến trúc Cache phức tạp hơn và vận dụng vào các hệ thống vi xử lý trong tương lai.

TÀI LIỆU THAM KHẢO

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, Morgan Kaufmann, 2017.

[2] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed., Morgan Kaufmann, 2017.

[3] W. Stallings, Computer Organization and Architecture: Designing for Performance, 10th ed., Pearson, 2016.

[4] D. M. Harris and S. L. Harris, Digital Design and Computer Architecture, 2nd ed., Morgan Kaufmann, 2012.

[5] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[6] Intel Corporation, ModelSim – Intel FPGA Edition Software User Guide, 2020.